Списки в Python: работа со структурой данных

Индексация, распаковка, срезы, управление элементами и особенности изменяемых типов

Создание и свойства списков

Список (list) — упорядоченная изменяемая коллекция объектов произвольных типов.

Квадратные скобки

Стандартный литерал `[1, 2, 3]`.

Конструктор list()

Преобразование итерируемых объектов.

Разнотипность

Хранение любых типов данных вместе.

Индексация: прямой и обратный доступ

```
# Прямой и отрицательный индексы
nums = [10, 20, 30, 40]
print(nums[0])   # 10 (первый)
print(nums[-1])  # 40 (последний)
# Изменение элемента на месте
nums[1] = 99
print(nums)      # [10, 99, 30, 40]
```

Нумерация с нуля и конца

Положительные индексы нумеруются слева направо начиная с 0. Отрицательные индексы отсчитываются справа налево начиная с -1.

> **Выход за границы вызывает IndexError**

Распаковка списков

Множественное присваивание

Число переменных слева должно строго совпадать с количеством элементов распаковываемого списка.

Несовпадение длины порождает ValueError

```
# Извлечение значений в переменные
user = ["Tom", 25, True]

name, age, is_active = user

print(name)      # Tom
print(age)       # 25
print(is_active) # True
```

Перебор элементов в цикле

Прямой обход (for item in list)

```
for item in users:
    print(item)
# Простой и рекомендуемый синтаксис
```

Обход по индексу (while / len)

```
i = 0
while i < len(users):
    print(users[i])
    i += 1
# Контроль индекса и позиции шага
```

Добавление: append, insert, extend

```
items = ["Tom", "Bob"]

# Добавление в конец
items.append("Alice")

# Вставка по индексу
items.insert(1, "Bill")

# Расширение другим списком
items.extend(["Sam", "Mike"])
```

Методы расширения коллекции

`append()` добавляет одиночный объект в конец. `insert()` сдвигает элементы вправо. `extend()` выполняет слияние списков.

> **Оператор + также объединяет списки**

Удаление: remove, pop, clear, del

Способы удаления значений

`remove()` удаляет по значению (первое совпадение). `pop()` удаляет по индексу и возвращает значение. Инструкция `del` удаляет по индексу или срезу.

> **Если значение не найдено, remove вызовет ValueError**

```
users = ["Tom", "Bob", "Alice", "Sam"]
users.remove("Bob")  # по значению
last = users.pop()   # удален "Sam"
del users[0]          # удален "Tom"
users.clear()         # список пуст: []
```

Срезы: синтаксис [start:end:step]

```
a = [10, 20, 30, 40, 50]
print(a[1:3])     # [20, 30]
print(a[:3])      # [10, 20, 30]
print(a[2:])      # [30, 40, 50]
print(a[::2])     # [10, 30, 50]
print(a[::-1])    # [50, 40, 30, 20, 10]
```

Получение подсписков

Конечный индекс end никогда не включается в срез. Срез всегда формирует независимый новый список.

a[::-1] — быстрый разворот списка

Поиск, подсчет и сортировка

Анализ и упорядочивание

Метод `sort()` сортирует на месте, а функция `sorted()` возвращает новую копию.

> **Оператор in проверяет вхождение элемента**

```
nums = [4, 2, 4, 1, 5]
print(nums.count(4))  # 2 (кол-во)
print(nums.index(2))  # 1 (индекс)
# Сортировка по возрастанию
nums.sort()           # [1, 2, 4, 4, 5]
# Реверс порядка элементов
nums.reverse()        # [5, 4, 4, 2, 1]
```

Мутабельность и поверхностное копирование

Присваивание ссылки (b = a)

```
a = [1, 2]
b = a         # тот же объект
b.append(3)
print(a)    # [1, 2, 3]
# a изменился вместе с b!
```

Копия (copy() или срез [:])

```
a = [1, 2]
b = a.copy()  # новый список
b.append(3)
print(a)      # [1, 2]
# a остался неизменным
```

Рекомендации по работе со списками

Проверка in перед index

Предотвращение исключения ValueError.

Явное копирование

Защита от побочных мутаций данных.

Прямая итерация

Использование `for item in list` без индексов.

Вопросы и обсуждение

Списки — фундаментальный изменяемый тип последовательностей в Python, лежащий в основе большинства алгоритмов